

LAB 02: CSS for Layout and Data Presentation

Web Application Development Course

1. LAB OVERVIEW

Course: Web Application Development

Target Students: 3rd–4th year Computer Science / IT students

Prerequisites: Lab 01 (HTML & HTTP Fundamentals)

Duration: 3 hours

Difficulty: Beginner → Intermediate

Lab Context

In Lab 01, students built structured HTML documents but everything looked like a 1990s website - plain text, default fonts, no colors. This is intentional. Now that students understand document structure and HTTP mechanics, they're ready to learn presentation.

This lab focuses on **CSS for real-world data presentation**, not artistic design. Students will learn to:

- Present tabular data effectively
- Create readable forms
- Build responsive layouts
- Style components consistently

Critical principle: CSS is about communication, not decoration. Good CSS makes information easier to understand.

What Students Will Build

Students will transform their Lab 01 websites by: 1. Creating a professional, readable layout using CSS Grid and Flexbox 2. Styling tables to present data clearly 3. Making forms user-friendly and accessible 4. Implementing responsive design (mobile-first approach) 5. Building reusable component styles

Students will **NOT** use CSS frameworks (Bootstrap, Tailwind, etc.). They must understand the fundamentals first.

Key Differences from Lab 01

Lab 01	Lab 02
Structure only	Structure + Presentation
Semantic meaning	Visual hierarchy
HTTP mechanics	CSS cascade, specificity
DevTools Network tab	DevTools Elements tab + Computed styles
Document thinking	Component thinking

2. LEARNING OBJECTIVES

By the end of this lab, students will be able to:

Technical Skills

- Write well-structured CSS following naming conventions
- Use CSS selectors effectively (element, class, ID, descendant, pseudo-classes)
- Implement layouts using Flexbox and CSS Grid
- Style forms for usability and accessibility
- Make tables readable and scannable
- Create responsive designs using media queries
- Debug CSS issues using browser DevTools

System Thinking Skills

- Understand the CSS cascade and specificity rules
 - Distinguish between layout, components, and utilities
 - Reason about separation of concerns (HTML structure vs CSS presentation)
 - Analyze performance implications of CSS selectors
 - Make informed trade-offs between flexibility and simplicity
 - Understand why inline styles and `!important` should be avoided
 - Prepare for JavaScript DOM manipulation (changing classes dynamically)
-

3. THEORY & CONCEPTS (GUIDE)

3.1 Core Concepts

What is CSS?

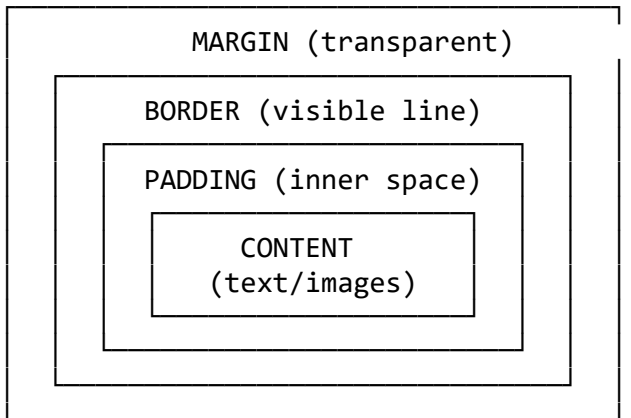
CSS (Cascading Style Sheets) is a language for describing how HTML elements should be presented. It controls: - Visual formatting (colors, fonts, spacing) - Layout (positioning, sizing, arrangement) - Responsiveness (different styles for different screen sizes) - Interactivity (hover effects, transitions)

Key principle: CSS separates presentation from structure. One HTML document can have many different “looks” by swapping CSS files.

HTML = What is this? (A heading, a paragraph, a list)
CSS = How should it look? (Large, blue, centered)

The Box Model

Every HTML element is a rectangular box with four layers:



Example:

```
.box {  
  width: 200px;           /* Content width */  
  padding: 20px;          /* Inner space */  
  border: 2px solid black; /* Border */  
  margin: 10px;           /* Outer space */  
}
```

```
/* Total width = 200 + 20*2 + 2*2 + 10*2 = 264px */
```

Critical understanding: When you set `width: 200px`, you're setting the **content** width, not the total element width. This causes layout bugs if you don't account for padding and border.

Solution: Use `box-sizing: border-box`

```
* {  
  box-sizing: border-box; /* Width includes padding and border */  
}  
  
.box {  
  width: 200px; /* Now total width is 200px, including padding/border */  
}
```

3.2 CSS Selectors and Specificity

Selector Types

```
/* Element selector - selects all <p> */
```

```
p {  
  color: black;  
}
```

```
/* Class selector - selects elements with class="intro" */
```

```
.intro {  
  font-size: 18px;  
}
```

```
/* ID selector - selects element with id="header" */
```

```

#header {
    background: blue;
}

/* Descendant selector - selects <a> inside <nav> */
nav a {
    color: white;
}

/* Child selector - selects direct children only */
nav > ul > li {
    display: inline-block;
}

/* Pseudo-class - selects based on state */
a:hover {
    color: red;
}

input:focus {
    border-color: blue;
}

tr:nth-child(even) {
    background: #f0f0f0;
}

/* Pseudo-element - selects part of element */
p::first-line {
    font-weight: bold;
}

/* Attribute selector */
input[type="email"] {
    border-color: green;
}

/* Multiple selectors */
h1, h2, h3 {
    font-family: Arial, sans-serif;
}

```

Specificity Rules

When multiple CSS rules target the same element, specificity determines which wins:

Specificity calculation:

Inline style:	1000 points
ID selector:	100 points
Class/attribute:	10 points
Element:	1 point

Examples:

```

/* Specificity: 1 (element) */
p {
    color: black;
}

/* Specificity: 10 (class) - WINS */
.intro {
    color: blue;
}

/* Specificity: 100 (ID) - WINS */
#special {
    color: red;
}

/* Specificity: 111 (1 ID + 1 class + 1 element) - WINS */
#header .nav a {
    color: white;
}

/* Specificity: 1000 (inline) - WINS */
<p style="color: green;">

```

Critical principle: Avoid high specificity. Prefer classes over IDs. Never use !important unless absolutely necessary (overriding third-party CSS).

Why? High specificity makes CSS hard to maintain. You end up in “specificity wars” where you keep adding more selectors to override previous rules.

Good practice:

```

/* Use single class names */
.button { }
.button-primary { }
.button-large { }

```

Bad practice:

```

/* Too specific, hard to override */
#header nav ul li a.active { }

```

3.3 The Cascade

CSS stands for **Cascading** Style Sheets. The cascade determines which styles apply when there are conflicts:

Order of precedence (highest to lowest): 1. Inline styles (style attribute) 2. Internal stylesheet (<style> in <head>) 3. External stylesheet (<link rel="stylesheet">) 4. Browser default styles

Within each level, specificity determines the winner.

If specificity is equal, the last rule wins:

```

p { color: red; }
p { color: blue; } /* This wins - appears later */

```

Important inheritance rules: - Some properties inherit from parent to child (color, font-family, text-align) - Some properties do NOT inherit (border, padding, margin, width, height)

```
/* These inherit */
body {
  color: #333;           /* All text will be #333 */
  font-family: Arial;    /* All text will use Arial */
}

/* These do NOT inherit */
div {
  border: 1px solid black; /* Only the div has border, not children */
  padding: 20px;           /* Only the div has padding */
}
```

3.4 Layout Systems

Flexbox - One-Dimensional Layout

Flexbox arranges elements in a **single direction** (row or column).

Use for:

- Navigation bars
- Card layouts
- Centering elements
- Distributing space between items
- Simple responsive layouts

```
.container {
  display: flex;
  justify-content: space-between; /* Horizontal alignment */
  align-items: center;           /* Vertical alignment */
  gap: 20px;                     /* Space between items */
}

/* Common patterns */
.nav {
  display: flex;
  gap: 1rem;
}

.card-container {
  display: flex;
  flex-wrap: wrap; /* Wrap to multiple lines */
  gap: 2rem;
}

.centered {
  display: flex;
  justify-content: center;
  align-items: center;
}
```

Flex properties (on container):

- flex-direction: row | column | row-reverse | column-reverse
- justify-content: flex-start | center | flex-end | space-between | space-around
- align-items: flex-start | center | flex-end | stretch - gap: spacing between items

Flex properties (on items):

- flex-grow: how much item should grow (0 = don't grow)
- flex-shrink: how much item should shrink (1 = can shrink)
- flex-basis: initial size before growing/shrinking

```
.item {  
    flex: 1; /* Shorthand for flex-grow: 1; flex-shrink: 1; flex-basis: 0; */  
}
```

CSS Grid - Two-Dimensional Layout

Grid arranges elements in **rows AND columns**.

Use for:

- Page layouts (header, sidebar, main, footer)
- Complex data displays
- Magazine-style layouts
- When you need precise control over rows and columns

```
.grid-container {  
    display: grid;  
    grid-template-columns: 200px 1fr 1fr; /* 3 columns: fixed, flexible, flexible */  
    grid-template-rows: auto 1fr auto; /* 3 rows: auto, flexible, auto */  
    gap: 20px;  
}
```

```
/* Common layout pattern */  
.page-layout {  
    display: grid;  
    grid-template-areas:  
        "header header header"  
        "sidebar main main"  
        "footer footer footer";  
    grid-template-columns: 200px 1fr 1fr;  
    grid-template-rows: auto 1fr auto;  
    min-height: 100vh;  
}
```

```
.header { grid-area: header; }  
.sidebar { grid-area: sidebar; }  
.main { grid-area: main; }  
.footer { grid-area: footer; }
```

Grid vs Flexbox Decision:

Use Flexbox when:

- Single row or column
- Content determines layout
- Need simple alignment

Use Grid when:

- Multiple rows AND columns
- Layout determines content placement
- Need precise control

3.5 Responsive Design

Mobile-First Approach

Critical principle: Design for mobile first, then enhance for larger screens.

Why?

- Forces you to prioritize content
- Easier to add features than remove them
- Better performance on mobile devices
- Mobile traffic often exceeds desktop

```
/* Base styles - Mobile (320px+) */
.container {
  padding: 1rem;
}

.grid {
  display: grid;
  grid-template-columns: 1fr; /* Single column on mobile */
  gap: 1rem;
}

/* Tablet (768px+) */
@media (min-width: 768px) {
  .container {
    padding: 2rem;
  }

  .grid {
    grid-template-columns: 1fr 1fr; /* Two columns */
  }
}

/* Desktop (1024px+) */
@media (min-width: 1024px) {
  .container {
    padding: 3rem;
    max-width: 1200px;
    margin: 0 auto;
  }

  .grid {
    grid-template-columns: repeat(3, 1fr); /* Three columns */
  }
}

/* Large desktop (1440px+) */
@media (min-width: 1440px) {
```

```

    .grid {
      grid-template-columns: repeat(4, 1fr); /* Four columns */
    }
  }
}

```

Common Breakpoints

```

/* Mobile:      320px - 767px (base styles, no media query) */
/* Tablet:      768px - 1023px */
/* Desktop:     1024px - 1439px */
/* Large:       1440px+ */

```

Important: These are guidelines. Choose breakpoints based on your **content**, not arbitrary device sizes.

3.6 Styling Forms for Usability

Forms are critical for web applications. Good form styling improves usability and reduces errors.

```

/* Make labels easy to read */
label {
  display: block;
  margin-bottom: 0.5rem;
  font-weight: 500;
}

/* Make inputs large enough to tap on mobile */
input,
select,
textarea {
  width: 100%;
  padding: 0.75rem;
  font-size: 16px; /* Prevents zoom on iOS */
  border: 1px solid #ccc;
  border-radius: 4px;
}

/* Visual feedback on focus */
input:focus,
select:focus,
textarea:focus {
  outline: none;
  border-color: #0066cc;
  box-shadow: 0 0 0 3px rgba(0, 102, 204, 0.1);
}

/* Error states */
input.error {
  border-color: #d32f2f;
}

input.error:focus {
  box-shadow: 0 0 0 3px rgba(211, 47, 47, 0.1);
}

/* Success states */
input.success {

```

```

    border-color: #388e3c;
}

/* Disabled states */
input:disabled {
    background-color: #f5f5f5;
    cursor: not-allowed;
}

/* Buttons */
button,
input[type="submit"] {
    padding: 0.75rem 1.5rem;
    font-size: 16px;
    background-color: #0066cc;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
}

button:hover {
    background-color: #0052a3;
}

button:active {
    background-color: #003d7a;
}

button:disabled {
    background-color: #ccc;
    cursor: not-allowed;
}

```

3.7 Styling Tables for Data Presentation

Tables display structured data. Good table styling makes data scannable and comparable.

```

table {
    width: 100%;
    border-collapse: collapse; /* Remove spacing between cells */
}

/* Header styling */
thead th {
    background-color: #f0f0f0;
    padding: 1rem;
    text-align: left;
    font-weight: 600;
    border-bottom: 2px solid #333;
}

/* Cell styling */
tbody td {
    padding: 0.75rem 1rem;
}

```

```

    border-bottom: 1px solid #e0e0e0;
}

/* Zebra striping for readability */
tbody tr:nth-child(even) {
    background-color: #f9f9f9;
}

/* Hover effect */
tbody tr:hover {
    background-color: #e3f2fd;
}

/* Numeric columns - right align */
td.numeric {
    text-align: right;
    font-variant-numeric: tabular-nums; /* Monospace numbers */
}

/* Status indicators */
td.status-active {
    color: #388e3c;
    font-weight: 500;
}

td.status-inactive {
    color: #757575;
}

/* Responsive table - scroll on mobile */
.table-container {
    overflow-x: auto;
}

@media (max-width: 768px) {
    table {
        font-size: 14px;
    }

    th, td {
        padding: 0.5rem;
    }
}

```

3.8 CSS Architecture and Naming

BEM (Block, Element, Modifier) - Recommended Naming Convention:

```

/* Block - standalone component */
.card { }

/* Element - part of block */
.card__title { }
.card__body { }
.card__footer { }

```

/ Modifier - variation of block or element */*

```
.card--featured { }  
.card--large { }  
.card__title--primary { }
```

Example:

```
<div class="card card--featured">  
  <h3 class="card__title card__title--primary">Featured Course</h3>  
  <div class="card__body">  
    <p>Content here...</p>  
  </div>  
  <div class="card__footer">  
    <button class="button button--primary">Enroll</button>  
  </div>  
</div>
```

```
.card {  
  border: 1px solid #e0e0e0;  
  border-radius: 8px;  
  padding: 1.5rem;  
}
```

```
.card--featured {  
  border-color: #0066cc;  
  border-width: 2px;  
}
```

```
.card__title {  
  margin: 0 0 1rem 0;  
  font-size: 1.5rem;  
}
```

```
.card__title--primary {  
  color: #0066cc;  
}
```

```
.button {  
  padding: 0.5rem 1rem;  
  background: #666;  
  color: white;  
  border: none;  
  cursor: pointer;  
}
```

```
.button--primary {  
  background: #0066cc;  
}
```

Why BEM?

- Class names are self-documenting
- Low specificity (all selectors are single classes)
- No naming conflicts
- Easy to understand component structure

3.9 Performance Considerations

CSS Selector Performance

Browsers read selectors **right to left**.

```
/* Slow - browser checks ALL <a>, then filters by .nav */  
.nav a { }
```

```
/* Faster - specific class */  
.nav-link { }
```

```
/* Very slow - checks ALL elements */  
* { }
```

```
/* Slow - complex nesting */  
#header nav ul li a { }
```

Best practices:

- Avoid universal selector (*) except for resets
- Avoid deep nesting (max 3 levels)
- Prefer classes over complex selectors
- Avoid descendant selectors in large documents

CSS File Organization

```
/* 1. Reset/Normalize */  
* { box-sizing: border-box; }  
body { margin: 0; }
```

```
/* 2. Variables (CSS custom properties) */  
:root {  
  --color-primary: #0066cc;  
  --color-text: #333;  
  --spacing-unit: 8px;  
}
```

```
/* 3. Typography */  
body { font-family: Arial, sans-serif; }  
h1, h2, h3 { /* ... */ }
```

```
/* 4. Layout */  
.container { /* ... */ }  
.grid { /* ... */ }
```

```
/* 5. Components */  
.card { /* ... */ }  
.button { /* ... */ }  
.form { /* ... */ }
```

```
/* 6. Utilities */  
.text-center { text-align: center; }  
.mt-2 { margin-top: 16px; }
```

```
/* 7. Media queries at the end */
@media (min-width: 768px) { /* ... */ }
```

3.10 Why This Matters in Real-World Systems

1. **Maintenance:** Well-structured CSS is easier to update. One change to a component class updates all instances.
 2. **Performance:** Inefficient CSS slows down rendering, especially on mobile devices.
 3. **Accessibility:** Proper color contrast, focus styles, and responsive sizing improve usability for all users.
 4. **Collaboration:** CSS naming conventions and architecture allow teams to work together without conflicts.
 5. **Preparation for JavaScript:** Understanding classes and selectors is essential for DOM manipulation (next lab).
 6. **Career:** Every frontend framework (React, Vue, Angular) requires solid CSS knowledge. You cannot skip fundamentals.
-

4. HANDS-ON GUIDE

4.1 Setup and File Structure

Recommended structure:

```
my-site/
├── index.html
├── courses.html
├── about.html
├── css/
│   └── styles.css
├── images/
│   └── (images)
```

Link CSS in HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Site</title>
  <link rel="stylesheet" href="css/styles.css">
</head>
<body>
  <!-- Content -->
</body>
</html>
```

4.2 Step-by-Step: Creating Basic Styles

Step 1: CSS Reset and Base Styles

Create `css/styles.css`:

```
/* =====  
CSS RESET  
===== */  
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}  
  
/* =====  
CSS VARIABLES  
===== */  
:root {  
  /* Colors */  
  --color-primary: #0066cc;  
  --color-primary-dark: #0052a3;  
  --color-text: #333333;  
  --color-text-light: #666666;  
  --color-border: #e0e0e0;  
  --color-background: #ffffff;  
  --color-background-alt: #f5f5f5;  
  
  /* Spacing */  
  --spacing-xs: 0.25rem; /* 4px */  
  --spacing-sm: 0.5rem; /* 8px */  
  --spacing-md: 1rem; /* 16px */  
  --spacing-lg: 1.5rem; /* 24px */  
  --spacing-xl: 2rem; /* 32px */  
  
  /* Typography */  
  --font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Arial, sans-serif;  
  --font-size-base: 16px;  
  --font-size-sm: 0.875rem;  
  --font-size-lg: 1.125rem;  
  --font-size-xl: 1.5rem;  
  --font-size-2xl: 2rem;  
  
  /* Layout */  
  --container-width: 1200px;  
  --border-radius: 4px;  
}  
  
/* =====  
BASE TYPOGRAPHY  
===== */  
body {  
  font-family: var(--font-family);  
  font-size: var(--font-size-base);  
  line-height: 1.6;
```

```

    color: var(--color-text);
    background-color: var(--color-background);
}

h1, h2, h3, h4, h5, h6 {
    line-height: 1.3;
    margin-bottom: var(--spacing-md);
}

h1 { font-size: var(--font-size-2xl); }
h2 { font-size: var(--font-size-xl); }
h3 { font-size: var(--font-size-lg); }

p {
    margin-bottom: var(--spacing-md);
}

a {
    color: var(--color-primary);
    text-decoration: none;
}

a:hover {
    text-decoration: underline;
}

```

Test: Refresh your HTML page. Everything should now have no default margins/padding and use consistent fonts.

Step 2: Layout Structure

Add to css/styles.css:

```

/* =====
   LAYOUT
   ===== */
.container {
    max-width: var(--container-width);
    margin: 0 auto;
    padding: 0 var(--spacing-md);
}

/* Header */
.site-header {
    background-color: var(--color-primary);
    color: white;
    padding: var(--spacing-lg) 0;
}

.site-header h1 {
    margin: 0;
    font-size: var(--font-size-2xl);
}

/* Navigation */

```

```

.site-nav {
    background-color: var(--color-primary-dark);
}

.site-nav ul {
    list-style: none;
    display: flex;
    gap: var(--spacing-md);
    padding: var(--spacing-md);
}

.site-nav a {
    color: white;
    padding: var(--spacing-sm) var(--spacing-md);
    border-radius: var(--border-radius);
}

.site-nav a:hover {
    background-color: rgba(255, 255, 255, 0.1);
    text-decoration: none;
}

.site-nav a.active {
    background-color: rgba(255, 255, 255, 0.2);
}

/* Main content */
.site-main {
    padding: var(--spacing-xl) 0;
    min-height: 60vh;
}

/* Footer */
.site-footer {
    background-color: var(--color-background-alt);
    padding: var(--spacing-lg) 0;
    text-align: center;
    color: var(--color-text-light);
    font-size: var(--font-size-sm);
    margin-top: var(--spacing-xl);
}

```

Update your HTML:

```

<body>
  <header class="site-header">
    <div class="container">
      <h1>University Course Catalog</h1>
    </div>
  </header>

  <nav class="site-nav">
    <div class="container">
      <ul>
        <li><a href="index.html" class="active">Home</a></li>

```

```

        <li><a href="courses.html">Courses</a></li>
        <li><a href="about.html">About</a></li>
    </ul>
</div>
</nav>

<main class="site-main">
    <div class="container">
        <!-- Your content here -->
    </div>
</main>

<footer class="site-footer">
    <div class="container">
        <p>&copy; 2025 University. All rights reserved.</p>
    </div>
</footer>
</body>

```

Test: Your site now has a colored header, horizontal navigation, and consistent spacing.

Step 3: Component - Card

Add to css/styles.css:

```

/* =====
COMPONENTS - Card
===== */

.card {
    background: white;
    border: 1px solid var(--color-border);
    border-radius: var(--border-radius);
    padding: var(--spacing-lg);
    margin-bottom: var(--spacing-lg);
}

.card__title {
    margin: 0 0 var(--spacing-md) 0;
    color: var(--color-primary);
}

.card__meta {
    font-size: var(--font-size-sm);
    color: var(--color-text-light);
    margin-bottom: var(--spacing-md);
}

.card__content {
    margin-bottom: var(--spacing-md);
}

.card--featured {
    border-color: var(--color-primary);
    border-width: 2px;
}

```

```

/* Card grid layout */
.card-grid {
  display: grid;
  gap: var(--spacing-lg);
}

@media (min-width: 768px) {
  .card-grid {
    grid-template-columns: repeat(2, 1fr);
  }
}

@media (min-width: 1024px) {
  .card-grid {
    grid-template-columns: repeat(3, 1fr);
  }
}

```

HTML example:

```

<div class="card card--featured">
  <h3 class="card__title">CS301 - Web Application Development</h3>
  <div class="card__meta">
    <strong>Instructor:</strong> Dr. Smith |
    <strong>Schedule:</strong> Mon/Wed 10:00-11:30
  </div>
  <div class="card__content">
    <p>Comprehensive introduction to building modern web applications...</p>
  </div>
  <a href="courses.html#cs301" class="button">View Details</a>
</div>

```

Step 4: Forms

Add to css/styles.css:

```

/* =====
FORMS
===== */

.form-group {
  margin-bottom: var(--spacing-lg);
}

.form-label {
  display: block;
  margin-bottom: var(--spacing-sm);
  font-weight: 500;
}

.form-input,
.form-select,
.form-textarea {
  width: 100%;
  padding: var(--spacing-md);
  font-size: var(--font-size-base);
}

```

```

    font-family: inherit;
    border: 1px solid var(--color-border);
    border-radius: var(--border-radius);
}

.form-input:focus,
.form-select:focus,
.form-textarea:focus {
    outline: none;
    border-color: var(--color-primary);
    box-shadow: 0 0 0 3px rgba(0, 102, 204, 0.1);
}

.form-textarea {
    min-height: 120px;
    resize: vertical;
}

/* Form Layout */
.form-row {
    display: grid;
    gap: var(--spacing-md);
}

@media (min-width: 768px) {
    .form-row {
        grid-template-columns: repeat(2, 1fr);
    }
}

/* Radio buttons and checkboxes */
.form-radio,
.form-checkbox {
    display: flex;
    align-items: center;
    margin-bottom: var(--spacing-sm);
}

.form-radio input,
.form-checkbox input {
    margin-right: var(--spacing-sm);
}

```

HTML example:

```

<form action="results.html" method="GET" class="form">
  <div class="form-row">
    <div class="form-group">
      <label for="name" class="form-label">Full Name</label>
      <input type="text" id="name" name="name" class="form-input" required>
    </div>

    <div class="form-group">
      <label for="email" class="form-label">Email</label>
      <input type="email" id="email" name="email" class="form-input" required>
    </div>
  </div>
</form>

```

```

        </div>
    </div>

    <div class="form-group">
        <label for="comments" class="form-label">Comments</label>
        <textarea id="comments" name="comments" class="form-textarea"></textarea>
    </div>

    <button type="submit" class="button button--primary">Submit</button>
</form>

```

Step 5: Tables

Add to css/styles.css:

```

/* =====
   TABLES
   ===== */
.table-container {
    overflow-x: auto;
    margin-bottom: var(--spacing-lg);
}

.table {
    width: 100%;
    border-collapse: collapse;
}

.table thead th {
    background-color: var(--color-background-alt);
    padding: var(--spacing-md);
    text-align: left;
    font-weight: 600;
    border-bottom: 2px solid var(--color-border);
}

.table tbody td {
    padding: var(--spacing-md);
    border-bottom: 1px solid var(--color-border);
}

.table tbody tr:nth-child(even) {
    background-color: #fafafa;
}

.table tbody tr:hover {
    background-color: #e3f2fd;
}

/* Table cell alignments */
.table td.numeric {
    text-align: right;
    font-variant-numeric: tabular-nums;
}

```

```

.table td.center {
    text-align: center;
}

/* Responsive table */
@media (max-width: 768px) {
    .table {
        font-size: var(--font-size-sm);
    }

    .table thead th,
    .table tbody td {
        padding: var(--spacing-sm);
    }
}

```

HTML example:

```

<div class="table-container">
    <table class="table">
        <thead>
            <tr>
                <th>Course Code</th>
                <th>Title</th>
                <th class="center">Credits</th>
                <th>Schedule</th>
                <th class="numeric">Seats Available</th>
            </tr>
        </thead>
        <tbody>
            <tr>
                <td>CS301</td>
                <td>Web Application Development</td>
                <td class="center">3</td>
                <td>Mon/Wed 10:00-11:30</td>
                <td class="numeric">15</td>
            </tr>
            <!-- More rows -->
        </tbody>
    </table>
</div>

```

Step 6: Buttons

Add to css/styles.css:

```

/* =====
   BUTTONS
   ===== */
.button {
    display: inline-block;
    padding: var(--spacing-sm) var(--spacing-lg);
    font-size: var(--font-size-base);
    font-weight: 500;
    text-align: center;
    border: none;
}

```

```

    border-radius: var(--border-radius);
    cursor: pointer;
    transition: background-color 0.2s ease;
    background-color: #666;
    color: white;
}

.button:hover {
    background-color: #555;
    text-decoration: none;
}

.button--primary {
    background-color: var(--color-primary);
}

.button--primary:hover {
    background-color: var(--color-primary-dark);
}

.button--large {
    padding: var(--spacing-md) var(--spacing-xl);
    font-size: var(--font-size-lg);
}

.button--block {
    display: block;
    width: 100%;
}

.button:disabled {
    background-color: #ccc;
    cursor: not-allowed;
}

```

4.3 Using Browser DevTools for CSS

Elements Tab

1. Right-click on any element → “Inspect”
2. See the HTML structure in left panel
3. See CSS rules in right panel

What to observe: - Which CSS rules are applied - Which rules are crossed out (overridden) - Computed values (final calculated values)

Live Editing CSS

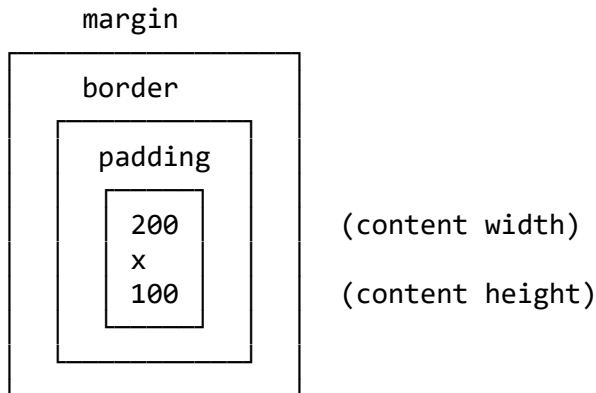
1. In Elements tab, click on any CSS rule
2. Change values (e.g., change color: blue to color: red)
3. See changes immediately in browser
4. **Important:** Changes are temporary. Copy changes to your CSS file to keep them.

Computed Tab

Shows the final computed values for all properties: - Click on element - Go to “Computed” tab - See exact pixel values, colors, etc.

Box Model Visualization

In Elements tab → Styles panel, scroll to bottom to see box model diagram:



Click on numbers to see which CSS rule set them.

Testing Responsive Design

1. Open DevTools
2. Click “Toggle device toolbar” (Ctrl+Shift+M)
3. Choose device (iPhone, iPad, etc.)
4. Test your media queries

Important: Test at custom widths to see how layout adapts: - 375px (mobile) - 768px (tablet) - 1024px (desktop) - 1440px (large desktop)

5. EXERCISES

Exercise 1 – Basic: Style Your Lab 01 Website

Difficulty: Beginner

Estimated Time: 60 minutes

Task Description

Take your Department Website from Lab 01 and add CSS styling. Transform it from plain HTML into a professional-looking site.

Requirements

Visual Requirements: 1. Typography

- Choose a readable font family
- Set appropriate font sizes for headings (h1-h3)
- Set line height for readability (1.5-1.8)
- Ensure good color contrast (text vs background)

2. Layout

- Header with site title
- Horizontal navigation menu
- Main content area with maximum width (1200px)
- Footer with copyright info
- Consistent spacing throughout

3. Navigation

- Horizontal list (using Flexbox)
- Clear hover states
- Indicate active/current page

4. Content Styling

- Faculty members displayed as cards or in a clean list
- Contact information formatted clearly
- Proper spacing between sections

5. Responsive Design

- Must work on mobile (320px+)
- Navigation should adapt (vertical on mobile)
- Content should reflow properly

Technical Requirements

- External CSS file (not inline or internal)
- Use CSS variables for colors and spacing
- Use Flexbox for navigation
- Use at least 3 different selector types (element, class, pseudo-class)
- Proper class naming (BEM or consistent convention)
- Mobile-first media queries

Constraints

- NO CSS frameworks (Bootstrap, Tailwind, etc.)
- NO inline styles
- NO !important unless you can justify it
- Maximum 3 colors (plus shades/tints)

Hints

- Start with reset and variables
- Build layout structure first (header, nav, main, footer)
- Style components (cards, lists) second
- Add responsive behavior last
- Test in DevTools at different screen sizes

What You Must Explain

1. **Color Choice:** Why did you choose these colors? Did you check color contrast for accessibility?
2. **Layout Decisions:** Why did you use Flexbox for navigation? Where would Grid be better?
3. **Responsive Strategy:** What changes between mobile and desktop? Why?
4. **Debugging:** Show a screenshot of DevTools showing:

- Box model of one element
 - One CSS rule being overridden (crossed out)
-

Exercise 2 – Intermediate: Course Catalog with Data Tables

Difficulty: Intermediate

Estimated Time: 75 minutes

Task Description

Enhance your Course Registration System from Lab 01 with professional CSS styling, focusing on presenting tabular data effectively.

Requirements

Pages to Style:

1. Home Page

- Hero section with featured course
- Quick stats (total courses, departments, etc.) in a grid
- Call-to-action buttons

2. Course Catalog Page

- Search/filter form at top
- Course table with proper styling
- Minimum 10 courses in table
- Sortable appearance (even though sorting doesn't work yet)

3. Registration Form Page

- Multi-section form with clear visual hierarchy
- Field grouping (personal info, course selection, preferences)
- Proper error state styling (use `.error` class)
- Submit button that stands out

4. Confirmation Page

- Success message styling
- Summary of submitted data (if visible in URL)

Functional Requirements

Table Styling:

- Zebra striping (alternating row colors)
- Hover effect on rows
- Sticky header (stays visible when scrolling)
- Numeric columns right-aligned
- Column sorting indicators (visual only, using CSS)

Form Styling:

- Clear visual distinction between sections
- Focus states on all inputs
- Disabled state styling

- Required field indicators (asterisk or label)
- Button primary vs secondary styles

Responsive Behavior:

- Table scrolls horizontally on mobile
- Form changes to single column on mobile
- Navigation becomes vertical menu on mobile
- Stats grid adapts (4 cols → 2 cols → 1 col)

Incomplete Requirements (You Decide)

1. **Table Interaction:** How do you indicate which columns are “sortable”? (Arrows? Icons? Text? Remember, you can’t actually sort yet, but you can style the appearance)
2. **Form Validation Feedback:** Where should error messages appear? Inline below field? At top of form? Both? What color and styling?
3. **Mobile Table Strategy:** Tables are hard on mobile. Do you:
 - Make it scroll horizontally?
 - Hide less important columns?
 - Transform into cards?
 - Something else?
4. **Loading States:** How would you style a “Loading...” state for the table? (You won’t implement it, but design the CSS)

Advanced Challenge

Create a “dark mode” version using CSS variables:

```
/* Light mode (default) */
:root {
  --color-background: #ffffff;
  --color-text: #333333;
}

/* Dark mode */
@media (prefers-color-scheme: dark) {
  :root {
    --color-background: #1a1a1a;
    --color-text: #e0e0e0;
  }
}
```

Test by changing your OS dark mode setting.

Hints

- Use position: sticky for table header
- Use overflow-x: auto for responsive tables
- Group related form fields with <fieldset>
- Use CSS Grid for stats layout
- Test table with 100+ rows to see performance

What You Must Explain

1. Table Design Decisions

- Why zebra striping?
- Why hover effects?
- How did you handle numeric columns?
- Mobile strategy and trade-offs

2. Form Usability Analysis

- How did you indicate required fields?
 - How did you show error states?
 - Why did you choose specific colors for errors/success?
 - What happens on focus?
-

Exercise 3 – Advanced: Multi-Theme Research Lab Website

Difficulty: Intermediate → Advanced

Estimated Time: 120 minutes

Task Description

Transform your Research Lab Website from Lab 01 into a fully styled, professional site with multiple theme options and advanced CSS features.

Requirements

Core Styling (All Pages):

1. Advanced Layout

- Complex grid layout on Publications page (filters sidebar + results)
- Flexbox-based People page (faculty grid, student list)
- Research projects in card grid with images
- Sticky navigation that changes on scroll

2. Component Library Create reusable components:

- `.card` (with variations: `.card--project`, `.card--person`, `.card--publication`)
- `.button` (with variations: `.button--primary`, `.button--secondary`, `.button--outline`)
- `.badge` (for tags like “PhD Student”, “Active”, “Published”)
- `.alert` (for messages/notifications)

3. Advanced Tables

- Publications table with:
 - Fixed header
 - Expandable rows (CSS only, no JS)
 - Column that can be hidden on mobile
 - Visual indicators for publication type

4. Advanced Forms

- Application form with:
 - Multi-step visual indicator (progress bar using CSS)
 - File upload styling

- Character counter styling (you'll add JS later)
- Conditional field visibility (using :checked selector)

Theme System

Implement **THREE themes** using CSS variables:

Theme 1: Professional Blue (default)

```
:root {
  --color-primary: #0066cc;
  --color-background: #ffffff;
  --color-text: #333333;
}
```

Theme 2: Academic Green

```
[data-theme="green"] {
  --color-primary: #2e7d32;
  --color-background: #fafafa;
  --color-text: #1b5e20;
}
```

Theme 3: Dark Mode

```
[data-theme="dark"] {
  --color-primary: #64b5f6;
  --color-background: #121212;
  --color-text: #e0e0e0;
}
```

HTML for theme switching:

```
<body data-theme="default">
<!-- OR -->
<body data-theme="green">
<!-- OR -->
<body data-theme="dark">
```

Change the data-theme attribute manually to test different themes.

Advanced CSS Features Required

1. CSS Grid Advanced:

```
/* Publications page layout */
.publications-layout {
  display: grid;
  grid-template-columns: 250px 1fr;
  grid-template-areas:
    "sidebar main"
    "sidebar main";
  gap: 2rem;
}

.sidebar { grid-area: sidebar; }
.main { grid-area: main; }
```

```

@media (max-width: 768px) {
  .publications-layout {
    grid-template-columns: 1fr;
    grid-template-areas:
      "sidebar"
      "main";
  }
}

```

2. Advanced Selectors:

```

/* Style every 3rd project card differently */
.project-card:nth-child(3n) {
  /* ... */
}

/* Style the first letter of publication titles */
.publication-title::first-letter {
  /* ... */
}

/* Show file name next to file input when selected */
input[type="file"]:not(:placeholder-shown) + label::after {
  content: "File selected";
}

/* Expandable row (using checkbox hack) */
.expandable-trigger:checked ~ .expandable-content {
  display: block;
}

```

3. Transitions and Transforms:

```

.card {
  transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.card:hover {
  transform: translateY(-4px);
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
}

.button {
  transition: all 0.2s ease;
}

.button:active {
  transform: scale(0.95);
}

```

4. Custom Properties (CSS Variables) Advanced Usage:

```

:root {
  --spacing-unit: 8px;
}

```

```
.card {
  /* Use calc with variables */
  padding: calc(var(--spacing-unit) * 2);
  margin-bottom: calc(var(--spacing-unit) * 3);
}

.button--large {
  padding: calc(var(--spacing-unit) * 1.5) calc(var(--spacing-unit) * 3);
}
```

Edge Cases to Handle

1. Very Long Text:

```
/* Truncate with ellipsis */
.truncate {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}

/* Show full text on hover */
.truncate:hover {
  white-space: normal;
}
```

2. Empty States:

```
/* Style empty table */
tbody:empty::after {
  content: "No publications found";
  display: block;
  padding: 2rem;
  text-align: center;
  color: #999;
}
```

3. Print Styles:

```
@media print {
  .site-nav,
  .site-footer,
  .button {
    display: none;
  }

  body {
    font-size: 12pt;
    color: black;
    background: white;
  }

  a::after {
    content: " (" attr(href) ")";
  }
}
```

4. High Contrast Mode:

```
@media (prefers-contrast: high) {  
  :root {  
    --color-border: #000;  
  }  
  
  .button {  
    border: 2px solid currentColor;  
  }  
}
```

5. Reduced Motion:

```
@media (prefers-reduced-motion: reduce) {  
  * {  
    animation: none !important;  
    transition: none !important;  
  }  
}
```

Performance Challenges

1. Large Lists/Tables:

- Your publications table has 50+ rows
- Each row has hover effect
- Test scroll performance in DevTools

2. CSS File Size:

- Your CSS file should be under 50KB
- Audit for duplicate rules
- Use CSS variables to reduce repetition

3. Selector Efficiency:

- Audit your selectors in DevTools
- Identify overly complex selectors
- Refactor to use classes

Hints

- Build the default theme first, then extract colors to variables
- Use DevTools to test theme switching
- Create a “style guide” page showing all components
- Test print preview (Ctrl+P)
- Test with browser extensions for color blindness

What You Must Explain (Required Technical Report)

1. Architecture Documentation

- How did you structure your CSS file?
- What naming convention did you use?
- How did you organize components?
- How would another developer navigate your CSS?

2. Theme System Analysis

- Explain how CSS variables enable theme switching
- What are the limitations of this approach?
- How would you add a 4th theme?
- What happens to images/icons in dark mode?

3. Responsive Design Strategy

- Document all breakpoints and why you chose them
- What layout changes at each breakpoint?
- What content is hidden/shown on mobile?
- How did you test edge cases (very small/large screens)?

4. Advanced CSS Features For each advanced feature you used:

- :nth-child() selectors
- CSS Grid
- Transforms and transitions
- Custom properties
- Pseudo elements

Explain:

- Why you used it
- Browser compatibility concerns
- Alternative approaches
- Real world use cases

5. Performance Audit Using DevTools:

- How many CSS rules total?
- How many rules apply to a single element?
- Any unused CSS (Coverage tab)?
- Rendering performance on large tables?
- What optimizations did you make?

6. Accessibility Testing

- Color contrast ratios (use tool like WebAIM)
- Focus indicator visibility
- Keyboard navigation
- Screen reader testing (if possible)
- What improvements would you make?

7. Edge Cases Handling For each edge case:

- Very long titles
 - Missing data
 - Empty states
 - Print view
 - What CSS techniques did you use?
 - What problems remain unsolved?
-

6. COMMON MISTAKES

Mistake 1: Using Inline Styles

Wrong:

```
<div style="color: red; font-size: 20px; margin: 10px;">
  Content
</div>
```

Why it's wrong: - Cannot be reused - Hard to maintain (change 100 instances?) - Cannot be overridden without !important - No separation of concerns - Cannot be cached by browser

Right:

```
/* In CSS file */
.error-message {
  color: red;
  font-size: 20px;
  margin: 10px;
}

<div class="error-message">Content</div>
```

Mistake 2: Overly Specific Selectors

Wrong:

```
#header nav ul li a.active {
  color: blue;
}
/* Specificity: 122 (very hard to override) */
```

Right:

```
.nav-link--active {
  color: blue;
}
/* Specificity: 10 (easy to manage) */
```

Why it matters:

- High specificity creates maintenance nightmares
 - Forces you into “specificity wars”
 - Hard to override without !important
-

Mistake 3: Not Using box-sizing: border-box

Wrong:

```
.box {
  width: 200px;
  padding: 20px;
  border: 2px solid black;
```

```
}
/* Total width: 200 + 40 + 4 = 244px (unexpected!) */
```

Right:

```
* {
  box-sizing: border-box;
}

.box {
  width: 200px;
  padding: 20px;
  border: 2px solid black;
}
/* Total width: 200px (predictable) */
```

Mistake 4: Using Fixed Pixel Values Everywhere

Wrong:

```
.container {
  width: 960px;      /* Breaks on mobile */
  padding: 15px;
  font-size: 14px;
}
```

Right:

```
.container {
  max-width: 1200px; /* Responsive */
  width: 100%;
  padding: 1rem;     /* Scales with font size */
  font-size: 1rem;
}
```

Mistake 5: Ignoring Responsive Design

Wrong:

```
/* No media queries at all */
.sidebar {
  width: 300px;
  float: left;
}

.main {
  width: 900px;
  float: right;
}
```

On mobile: Content overflows, users must scroll horizontally (terrible UX).

Right:

```

/* Mobile first */
.sidebar,
.main {
  width: 100%;
}

@media (min-width: 768px) {
  .sidebar {
    width: 300px;
  }
  .main {
    width: calc(100% - 300px);
  }
}

```

Mistake 6: Poor Color Contrast

Wrong:

```

body {
  color: #999;           /* Light gray text */
  background: #eee;      /* Light gray background */
}
/* Contrast ratio: 1.5:1 (fails WCAG) */

```

Right:

```

body {
  color: #333;           /* Dark gray text */
  background: #fff;      /* White background */
}
/* Contrast ratio: 12.6:1 (passes WCAG AAA) */

```

Why it matters:

- Legal compliance (ADA, Section 508)
- Users with low vision cannot read content
- Fails accessibility audits

Test: Use WebAIM Contrast Checker (<https://webaim.org/resources/contrastchecker/>)

Mistake 7: Forgetting Focus Styles

Wrong:

```

*:focus {
  outline: none; /* Removes all focus indicators */
}

```

Why it's dangerous:

- Keyboard users cannot see where they are
- Violates accessibility guidelines
- Makes forms unusable for many users

Right:

```
/* Custom focus style */
input:focus,
button:focus {
  outline: 2px solid #0066cc;
  outline-offset: 2px;
}

/* Or use browser default (don't remove it) */
```

Mistake 8: Using !important Everywhere

Wrong:

```
.text-red {
  color: red !important;
}

.button {
  background: blue !important;
}

.header {
  padding: 20px !important;
}
```

Why it's wrong:

- Creates specificity chaos
- Makes debugging extremely difficult
- Forces others to use !important to override
- Sign of poor CSS architecture

When !important IS acceptable:

- Overriding third-party CSS (e.g., Bootstrap)
 - Utility classes that should always win (.hidden { display: none !important; })
-

Mistake 9: Not Using CSS Variables

Wrong:

```
.header {
  background: #0066cc;
}

.button {
  background: #0066cc;
}

.link {
  color: #0066cc;
}
```

```
}

/* Later: Need to change brand color? Edit 50+ places */
```

Right:

```
:root {
  --color-primary: #0066cc;
}

.header {
  background: var(--color-primary);
}

.button {
  background: var(--color-primary);
}

.link {
  color: var(--color-primary);
}

/* Change brand color? Edit one line */
```

Mistake 10: Ignoring Mobile Performance

Wrong:

```
.background {
  background-image: url('huge-image-5mb.jpg');
}

.card:hover {
  box-shadow: 0 20px 50px rgba(0,0,0,0.5);
  transform: scale(1.1) rotate(2deg);
  filter: blur(5px);
  /* Heavy effects on mobile = poor performance */
}
```

Right:

```
/* Lighter image for mobile */
.background {
  background-image: url('small-image-100kb.jpg');
}

@media (min-width: 1024px) {
  .background {
    background-image: url('large-image-500kb.jpg');
  }
}

/* Simpler effects on mobile */
.card:hover {
```

```
    box-shadow: 0 4px 8px rgba(0,0,0,0.2);
}

@media (min-width: 1024px) {
    .card:hover {
        box-shadow: 0 20px 50px rgba(0,0,0,0.5);
        transform: scale(1.05);
    }
}
```

7. LAB COMPLETION CHECKLIST

CSS File Structure

- ☐ External CSS file (not inline or internal styles)
- ☐ Organized sections with comments
- ☐ CSS reset/normalize at top
- ☐ CSS variables defined in :root
- ☐ Logical order (reset → variables → base → layout → components → utilities → media queries)

Typography

- ☐ Readable font family chosen
- ☐ Appropriate font sizes (base size 16px recommended)
- ☐ Good line height (1.5-1.8 for body text)
- ☐ Proper heading hierarchy styling
- ☐ Color contrast meets WCAG AA (4.5:1 minimum for body text)

Layout

- ☐ Consistent spacing system (using variables)
- ☐ Maximum content width set
- ☐ Responsive at all breakpoints
- ☐ No horizontal scrolling on mobile
- ☐ Header, navigation, main, footer clearly defined

Components

- ☐ Cards styled consistently
- ☐ Buttons with clear hover states
- ☐ Forms with focus states
- ☐ Tables with proper styling
- ☐ All components use consistent class naming

Responsive Design

- ☐ Mobile-first approach
- ☐ Minimum 2 breakpoints defined
- ☐ Navigation adapts to mobile
- ☐ Images are responsive (max-width: 100%)
- ☐ Tables handle overflow on mobile

- ☐ Tested at 375px, 768px, 1024px, 1440px

Forms

- ☐ Proper spacing and alignment
- ☐ Focus states on all inputs
- ☐ Error states defined (even if not used yet)
- ☐ Labels associated with inputs
- ☐ Submit button stands out visually
- ☐ Form is keyboard accessible

Tables

- ☐ Proper spacing in cells
- ☐ Header styled differently from body
- ☐ Zebra striping OR hover effect
- ☐ Numeric columns right-aligned
- ☐ Responsive strategy implemented

Browser Compatibility

- ☐ Works in Chrome
- ☐ Works in Firefox
- ☐ Flexbox used correctly
- ☐ Grid used correctly (if applicable)
- ☐ No vendor prefixes needed for modern features

Performance

- ☐ CSS file under 100KB (preferably under 50KB)
- ☐ No duplicate rules
- ☐ Efficient selectors (avoid deep nesting)
- ☐ Images optimized

Accessibility

- ☐ Color contrast passes WCAG AA
- ☐ Focus states clearly visible
- ☐ No outline: none without replacement
- ☐ Text can be resized to 200% without breaking
- ☐ Keyboard navigation works

Code Quality

- ☐ Consistent indentation
- ☐ Meaningful class names
- ☐ Comments for complex sections
- ☐ No unused CSS
- ☐ Follows chosen naming convention (BEM recommended)

Testing

- ☐ Tested in DevTools at various screen sizes

- ☐ Inspected box model for key elements
 - ☐ Verified no CSS errors in console
 - ☐ Tested keyboard navigation
 - ☐ Tested with zoom (Ctrl/Cmd +)
-

8. THINKING & DISCUSSION QUESTIONS

Question 1: Specificity Wars

Scenario: You have this CSS:

```
.button {  
  background: gray;  
}  
  
.button-primary {  
  background: blue;  
}
```

And this HTML:

```
<button class="button button-primary">Click me</button>
```

The button is blue (correct).

But then someone adds:

```
nav button {  
  background: green;  
}
```

Now your primary button in nav is green (wrong).

Questions: a) Why does `nav button` override `.button-primary`?

- b) How would you fix this WITHOUT using `!important`?
 - c) How would you redesign the CSS to avoid this problem?
 - d) What does this teach you about CSS architecture?
-

Question 2: Flexbox vs Grid

Scenario: You're building a page with: - Header (full width) - Navigation bar (horizontal) - Main content area - Sidebar - Footer (full width)

Questions: a) Would you use Flexbox or Grid for the overall page layout? Why?

- b) Would you use Flexbox or Grid for the navigation bar? Why?
- c) What if the sidebar should be on the left on desktop but move below content on mobile?
- d) Can you combine Flexbox and Grid in the same page? Give an example.

Question 3: Mobile-First vs Desktop-First

Compare these two approaches:

Approach A: Desktop-First

```
.grid {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
}  
  
@media (max-width: 768px) {  
  .grid {  
    grid-template-columns: 1fr;  
  }  
}
```

Approach B: Mobile-First

```
.grid {  
  display: grid;  
  grid-template-columns: 1fr;  
}  
  
@media (min-width: 768px) {  
  .grid {  
    grid-template-columns: repeat(3, 1fr);  
  }  
}
```

Questions: a) What's the difference in how these work?

- b) What happens to a device that doesn't support media queries?
- c) Why is mobile-first considered better practice?
- d) Are there cases where desktop-first makes sense?

Question 4: Performance Investigation

Scenario: You have a table with 1000 rows. Each row has:

```
table tr:nth-child(even) {  
  background: #f0f0f0;  
}  
  
table tr:hover {  
  background: #e3f2fd;  
  transform: scale(1.01);  
  box-shadow: 0 4px 8px rgba(0,0,0,0.2);  
}
```

Questions: a) When you scroll the table, does it feel smooth or janky?

- b) Which CSS property is most likely causing performance issues?
 - c) How would you optimize this for better performance?
 - d) Use DevTools Performance tab - can you see the difference?
-

Question 5: CSS Variables vs Sass Variables

CSS Variables:

```
:root {  
  --color-primary: blue;  
}  
  
.button {  
  background: var(--color-primary);  
}  
  
[data-theme="dark"] {  
  --color-primary: lightblue;  
}
```

Sass Variables:

```
$color-primary: blue;  
  
.button {  
  background: $color-primary;  
}  
  
// Can't change at runtime
```

Questions: a) What's the key difference between CSS variables and Sass variables?

- b) What can CSS variables do that Sass variables cannot?
 - c) What can Sass variables do that CSS variables cannot?
 - d) When would you use each?
-

Question 6: The Cascade in Action

Given this CSS (in order):

```
/* 1 */ p { color: black; }  
/* 2 */ .intro { color: blue; }  
/* 3 */ p.intro { color: red; }  
/* 4 */ #special { color: green; }
```

And this HTML:

```
<p class="intro" id="special">What color is this?</p>
```

Questions: a) What color is the text? Why?

b) If you remove the id, what color would it be?

c) What if there's also an inline style: `<p style="color: yellow">`?

d) How would you calculate specificity for `p.intro.featured#special`?

Question 7: Real-World Color Systems

Scenario: Your design team gives you a color palette:

Primary: #0066cc

Secondary: #ff6b35

Success: #28a745

Warning: #ffc107

Error: #dc3545

But you need variations (hover states, disabled states, backgrounds, etc.).

Questions: a) How would you organize these colors using CSS variables?

b) How would you create lighter/darker versions without design tools?

c) What about transparency variations?

d) Design a system where changing ONE variable updates the entire color scheme.

Question 8: Debugging CSS Layout Issues

Scenario: You have this HTML:

```
<div class="container">
  <div class="box">Box 1</div>
  <div class="box">Box 2</div>
  <div class="box">Box 3</div>
</div>
```

And this CSS:

```
.container {
  display: flex;
}

.box {
  width: 300px;
}
```

On a 1200px screen, boxes display correctly. On a 900px screen, boxes overflow horizontally.

Questions: a) Why do boxes overflow instead of wrapping?

b) What's the minimum CSS change to fix this?

- c) How would you use DevTools to diagnose this?
 - d) What if you wanted boxes to shrink instead of wrap?
-

Question 9: Accessibility vs Aesthetics

Scenario: Your designer gives you this:

```
.fancy-button {  
  background: linear-gradient(45deg, #667eea 0%, #764ba2 100%);  
  color: white;  
  text-shadow: 0 1px 2px rgba(0,0,0,0.3);  
  border: none;  
  padding: 1rem 2rem;  
  font-size: 14px;  
}
```

Looks beautiful, but: - Text contrast ratio is 3.2:1 (fails WCAG) - No focus indicator - Small font size

Questions: a) How would you fix accessibility issues while keeping the design appealing?

- b) What compromises would you make?
 - c) How do you have this conversation with a designer?
 - d) What if the client insists on the original design?
-

Question 10: CSS in Large-Scale Applications

Scenario: You're joining a team with 20 developers working on a web application with 200+ pages.

Current CSS situation: - One 5000-line CSS file - Mix of naming conventions - Lots of !important - Duplicate rules - No variables - No documentation

Questions: a) What problems would you expect to encounter?

- b) How would you propose refactoring this?
 - c) What conventions/tools would you introduce?
 - d) How do you prevent this from happening again?
 - e) At what point would you consider a CSS-in-JS solution?
-

APPENDIX: Additional Resources

Tools

Color Tools: - WebAIM Contrast Checker: <https://webaim.org/resources/contrastchecker/> - Colors (palette generator): <https://colors.co/> - Color Hunt: <https://colorhunt.co/>

CSS Validation: - W3C CSS Validator: <https://jigsaw.w3.org/css-validator/>

Browser DevTools: - Chrome DevTools: <https://developer.chrome.com/docs/devtools/> - Firefox DevTools: <https://firefox-source-docs.mozilla.org/devtools-user/>

Learning Resources: - MDN CSS: <https://developer.mozilla.org/en-US/docs/Web/CSS> - CSS Tricks: <https://css-tricks.com/> - A Complete Guide to Flexbox: <https://css-tricks.com/snippets/css/a-guide-to-flexbox/> - A Complete Guide to Grid: <https://css-tricks.com/snippets/css/complete-guide-grid/>

Responsive Design Testing: - Responsive Design Checker: <https://responsivedesignchecker.com/> - Browser DevTools device emulation

End of Lab 02
